

Slam-Video Navigation-Final Submission

Students: Ron Keter, Yehonatan Hirschel

August 2, 2025

Source code is available at [Git Repository](#).

Contents

1	Introduction and Overview	2
2	Code	2
2.1	Triangulation	2
2.2	RANSAC	2
2.3	PnP Trajectory Calculation	3
2.4	Database Definition and Management	3
2.5	Single Bundle Creation	3
2.6	Relative Transformation and Covariance Extraction from Bundle Result	3
2.7	Pose Graph Building and Loop Closure Creation	3
3	Performance Analysis	4
3.1	Feature extraction and Matching	4
3.1.1	Inlier Count	4
3.1.2	Processing Time	4
3.1.3	Keypoint Spread	5
3.1.4	Repeatability	6
3.2	Tracking and RanSAC	7
3.2.1	Tracking statistics	7
3.2.2	Matching and Filtering Efficiency	9
3.3	Trajectory Estimation Performance	10
3.4	Bundle adjustment optimization analysis	11
3.4.1	Optimization Error - Mean factor vs keyframes	11
3.4.2	Median Projection Error	12
3.4.3	Projection Error vs. Distance from Reference Frame	13
3.5	Absolute Estimation Error	14
3.6	Relative Estimation Error	16
4	Discussion and Conclusions	20

1 Introduction and Overview

Simultaneous Localization and Mapping (SLAM) is a fundamental problem in the fields of robotics and computer vision. Its core objective is to construct a map of an unknown environment while concurrently estimating the pose (position and orientation) of a sensor within that environment. In the context of this project, which focuses on Vision Aided Navigation, the primary aim is to determine a vehicle's trajectory from video captured by an onboard stereo camera.

This problem is of significant importance for autonomous systems, as it enables localization and comprehensive environmental understanding using only visual input. This capability is critical in various real-world problems where reliance on external sensors, such as GPS, could be limited. For instance, in environments such as inside buildings GPS reception can be severely affected. Therefore, vision-based navigation is essential for ensuring continuous and accurate localization. Another significant use of a SLAM system is when higher accuracy than GPS is required, for example, in virtual augmented reality, or even in military defense systems.

Throughout the semester, we have incrementally developed a system to address these challenges. The development progressed through several key stages, each building on the previous to provide both local visual odometry and global trajectory estimation.

The entire system was implemented and evaluated using [KITTI stereo dataset](#), specifically a route within visual odometry sequence #5.

This report provides a detailed account of the various stages of our system, exploring their properties and providing a thorough analysis of the quality of the estimation we achieved.

2 Code

This section describes the main modules of our system.

2.1 Triangulation

`process_stereo_pair (alg.py , rows 140 -164 )`, `trangulate_inliers (alg.py, rows 62 - 91)`.

For every pair of left and right raw images we find matches of keypoints and perform triangulation to find their corresponding 3D points. We return only matches that have passed the following geometric tests:

- The left keypoint and the right keypoint share the same y coordinate with a maximum deviation of 2 pixels.
- The matching 3D point is not behind the camera, which is a clearly incorrect match.
- The matching 3D point is not too far from the camera measured by at least two pixels deviation in the x coordinates. If the point is too far, in later stages, this point leads to numerical issues.

2.2 RANSAC

We perform motion estimation only on matches that exist in all 4 images. We validate that the keypoint exists in all 4 images using (`find_stereo_temporal_matches`, `utility.py` rows 178 - 220).

Using the common matches, we perform a RANSAC algorithm to estimate the relative motion between frames (`perform_motion_estimation.py` rows 99 -117). In every RANSAC iteration we perform the following steps:

1. sample 4 keypoints.
2. perform a PnP motion estimation (`solve_pnp_and_locations.py`).
3. find the motion estimation's supporters (`find_ransac_iteration_supporters.py`).

Finally, we return the motion estimation that had the highest number of supporters.

2.3 PnP Trajectory Calculation

For every consecutive frame, we find the relative motion between the frames using RANSAC and then compose the relative motions to find the absolute matrix of the camera in the first camera coordinate system (alg.py row 264).

To calculate the PnP trajectory we extract the pose from each frame absolute matrix. An example can be seen in (plot_trajectories.py rows 96-100).

2.4 Database Definition and Management

We are using the TrackingDB (tracking_database.py) to save and manage our data from the matchings, triangulations, and PnP process. We initialize and import data to the DB in create_tracking_db (alg.py rows 196 - 270).

2.5 Single Bundle Creation

To perform bundle adjustment we take the whole PnP estimated trajectory and split the trajectory into separate windows (window_selector.py). For each window, we create a NonlinearFactorGraph (initialize_factor_graph_in_window , alg.py rows 373 - 405). After we create a factor graph, we find the optimized result for each graph and save it to a ba_results.pkl file.

2.6 Relative Transformation and Covariance Extraction from Bundle Result

To extract the relative transformation and covariance we load the ba_results.pkl file and compute the relative information (extract_relative_pose_covariance , pose_graph_loop_closure.py rows 73-104)

2.7 Pose Graph Building and Loop Closure Creation

(Build_pose_graph_with_loop_closure, pose_graph_loop_closure.py rows 110-166).

From the relative transformations and covariances that were computed in last module we build a pose graph with BetweenFactorPose3 factors. The initial estimate for each pose is the bundle adjustment results. To add a loop closure, we used the “cheat” presented in class. We peeked into the ground truth data of the last frame and added the corresponding measurement to the pose graph.

3 Performance Analysis

3.1 Feature extraction and Matching

This section analyzes the performance of different feature detectors and descriptors, SIFT, ORB, and AKAZE. The evaluation is based on the following metrics: inlier count, processing time, keypoint spread, and repeatability. These are fundamental to the accuracy and robustness of our system. The quality of feature extraction and matching directly influences the performance of downstream process, such as pose estimation, and the global optimization achieved.

AKAZE descriptors were selected for our system because of their optimal balance across these metrics for our project. Initially, we used ORB descriptors due to their fast computation time. However, as our system evolved, ORB presented performance issues, specifically leading to poor PnP estimations. To resolve this, we switched to AKAZE descriptors. While SIFT provided a slightly higher feature count, it had the highest processing time. AKAZE, in contrast, offered sufficient features, meeting our specified rejection policy, and provided a much better quality-time trade-off.

3.1.1 Inlier Count

An inlier is considered a keypoint / a 3D point that passed the geometric triangulation tests explained in 2.1. The average inlier count, a primary indicator of descriptor quality for accurate pose estimation, consistently showed AKAZE’s strong performance and consistency among SIFT, ORB, and AKAZE descriptors, as illustrated in Figure 1.



Figure 1: Average Inlier Count per Detector (detector_analysis.py, line 295).

Figure 1 illustrates the average number of inliers found by SIFT, ORB, and AKAZE descriptors, highlighting AKAZE’s strong performance and consistency.

3.1.2 Processing Time

Regarding processing efficiency, SIFT demonstrated the highest processing time. Conversely, ORB offered the fastest processing time. AKAZE provided a significant improvement over SIFT in terms of speed, offering a much better trade-off between time and the metrics described in this section, as depicted in Figure 2.

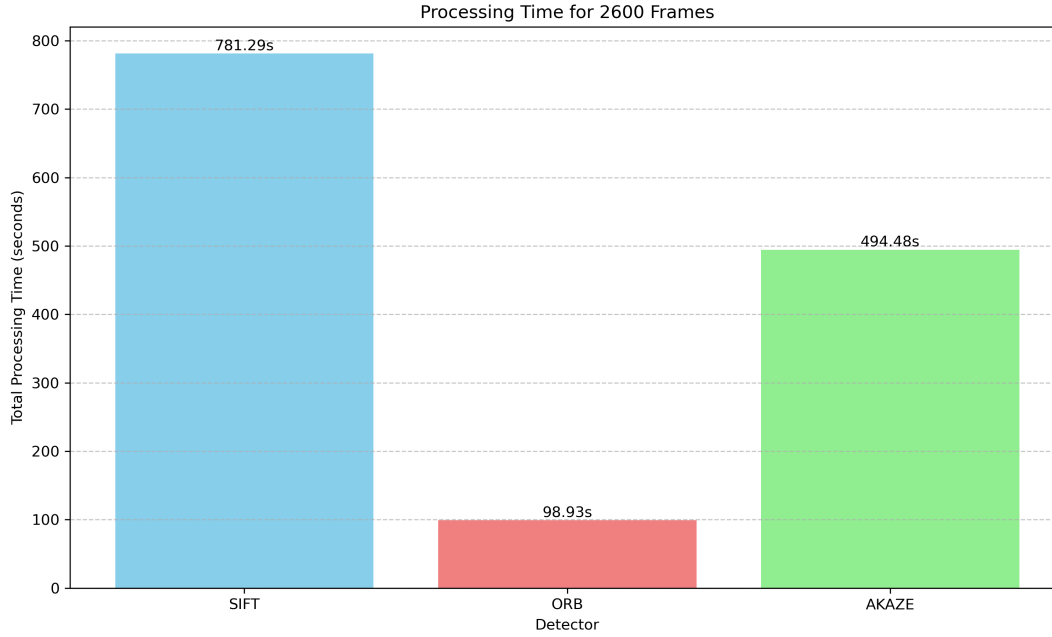


Figure 2: Cumulative processing time (detector_analysis.py, line 304).

Figure 2 presents the cumulative processing time for SIFT, ORB, and AKAZE across the sequence, clearly showing ORB’s speed advantage, and AKAZE being significantly faster than SIFT.

3.1.3 Keypoint Spread

A key consideration in selecting the appropriate detector was the spatial distribution of the detected keypoints across the image. A more even and widespread distribution allows for more robust and stable pose estimation, as it provides better geometric coverage of the scene.

To quantify keypoint spread, we divided the image into rectangular cells of 20 x 20 pixels and calculated the percentage of cells containing at least one keypoint detected by the detector. AKAZE demonstrated an average Keypoint Spread of 47.51% (Figure 3), which is significantly better than ORB, and slightly lower than SIFT. Figure 4 further illustrates the stability of keypoint distribution for each detector throughout the sequence.

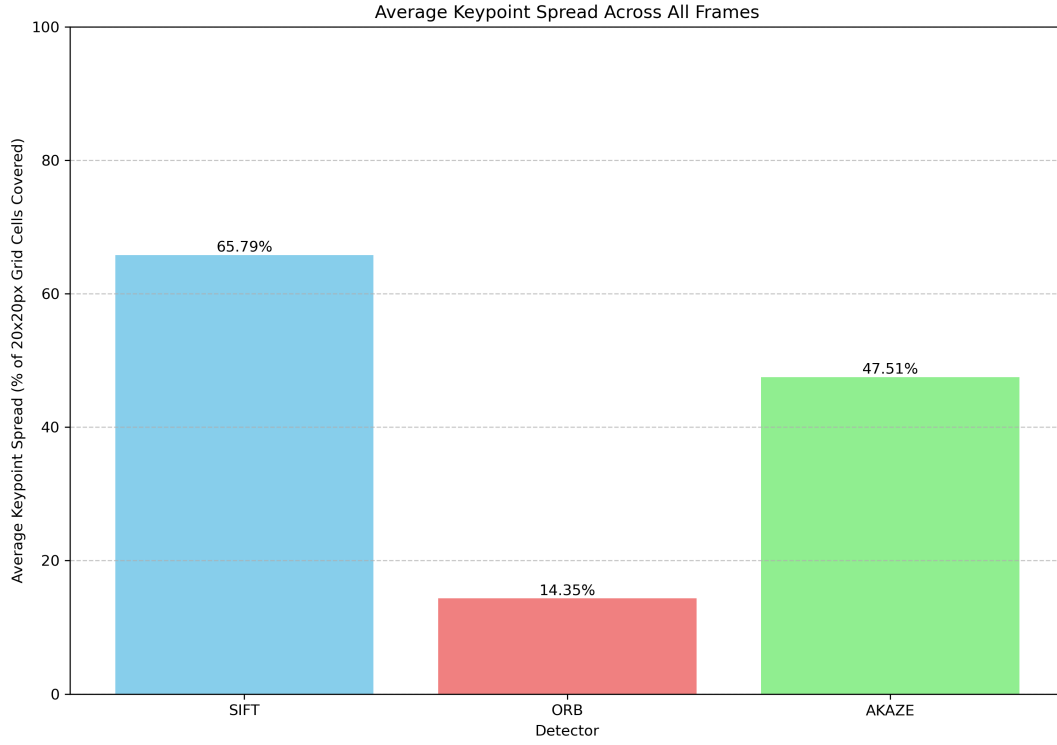


Figure 3: Average keypoint spread for each detector (detector_analysis.py, line 295).

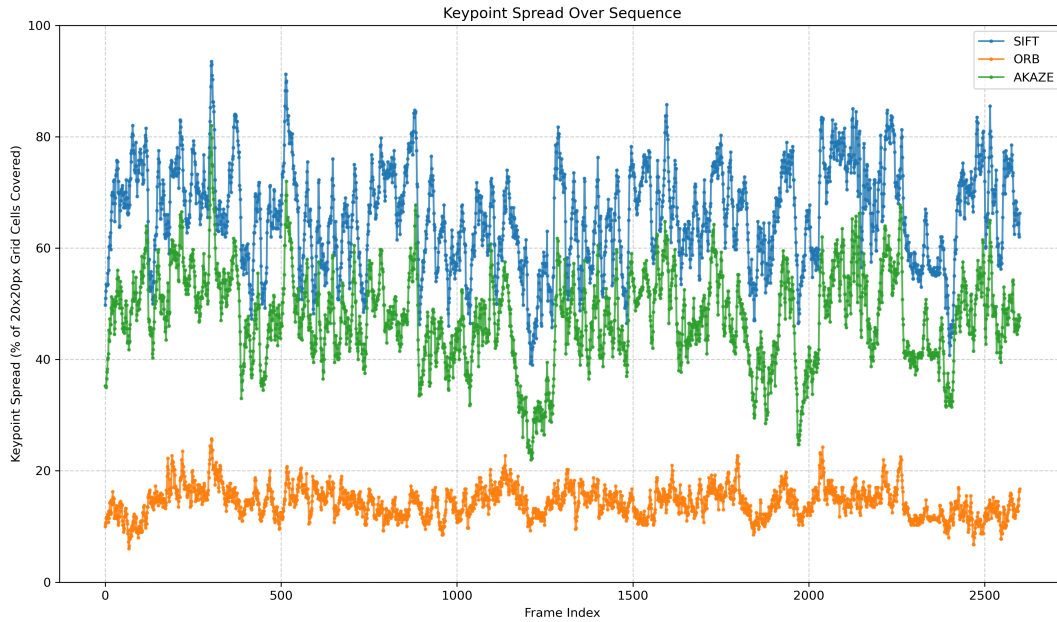


Figure 4: Keypoint spread over the image sequence (detector_analysis.py, line 295).

3.1.4 Repeatability

To further assess detector robustness, we introduced a repeatability test. This test aims to quantify how consistently a detector re-identifies a common set of keypoints across different images. This consistency is crucial for SLAM systems because feature points are the foundation for all subsequent processes. It ensures accurate, long-term feature tracking across frames, which is vital for reliable camera pose estimation using PnP with RANSAC. High repeatability also helps build a consistent 3D

map by allowing for the triangulation of the same features from multiple viewpoints over time. This, in turn, minimizes accumulated drift in the vehicle’s estimated trajectory, significantly improving overall accuracy.

For our evaluation, we applied random affine transformations (rotation, scale and translations) to an image and detected keypoints in both the original and transformed versions. A keypoint was considered ”repeatable” if its projected location in the warped image was within a 3-pixel threshold of a detected keypoint.

We defined the repeatability of detector in an image as the ratio of repeatable keypoints to the total number of keypoints in the original image. For each detector, we computed the mean and standard deviation of its repeatability across 100 randomly transformed image pairs.

As shown in Figure 5, ORB achieved the highest mean repeatability, followed closely by AKAZE. However, AKAZE demonstrated lower variability, indicated by a smaller standard deviation. In contrast, SIFT had the lowest repeatability among the tested detectors. These results suggest that both ORB and AKAZE are effective at consistently re-detecting features under affine transformations, with AKAZE offering slightly more stable performance. This robustness directly contributes to improved stability and accuracy in both the visual odometry and global optimization stages of our SLAM pipeline.

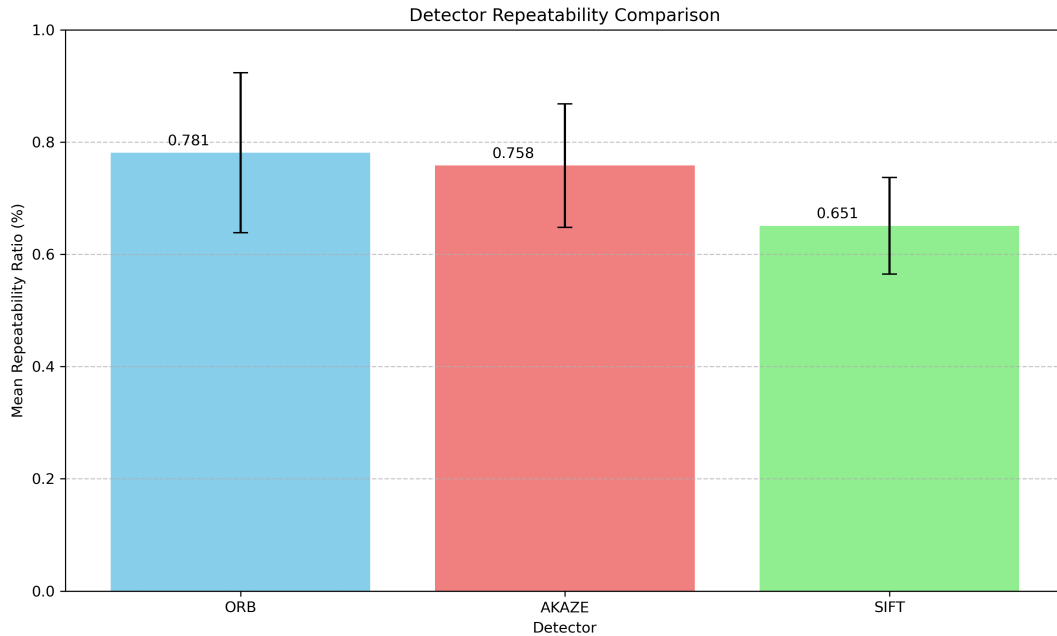


Figure 5: Detector repeatability comparison (detector_analysis.py, line 340).

3.2 Tracking and RanSAC

This section presents key statistics regarding the feature tracking process and evaluates the performance of matching and RANSAC based on our rejection policy.

3.2.1 Tracking statistics

- **Total number of tracks:** The system identified 287,146 individual feature tracks across all frames.
- **Total number of frames:** The dataset processed consists of 2,600 frames.
- **Mean track length:** On average, a feature track spans approximately 5.19 frames, with a minimum track length of 2 frames and a maximum of 163 frames.

- **Mean number of links per frame:** On average, there are 573.30 links associated with each frame.

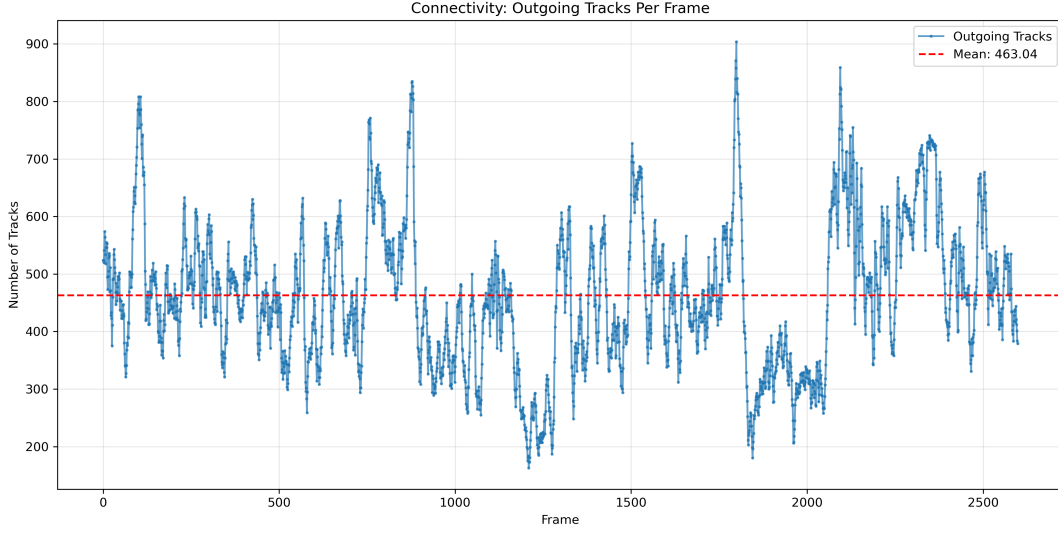


Figure 6: Database Connectivity (ransac_pnp_analysis.py, line 180).

Figure 6 illustrates the number of tracks that originate in a given frame and successfully continue into the subsequent frame. The horizontal dashed red line indicates the mean number of outgoing tracks. This graph is critical as it directly demonstrates the system’s ability to maintain continuity of features, which is crucial for robust visual odometry and consistent pose estimation over time.

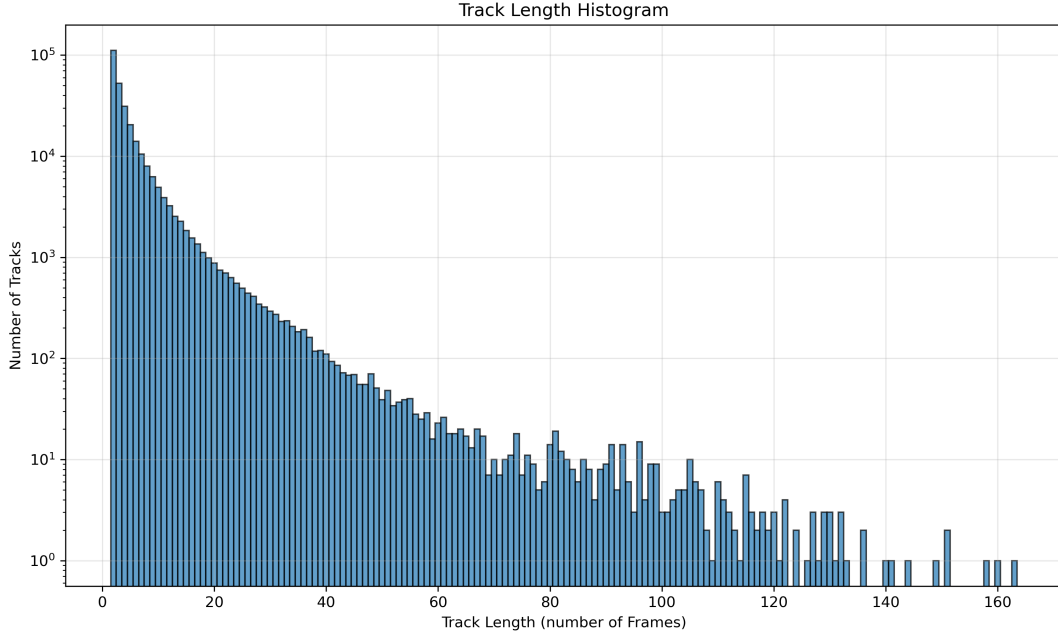


Figure 7: Track Length Histogram (ransac_pnp_analysis.py, line 183).

The histogram in figure 7 displays the distribution of feature track lengths, showing how many frames each track spans. This type of plot is crucial for understanding the persistence and stability of tracked features throughout the images sequence. Longer tracks contribute significantly to improved pose estimation accuracy and overall system robustness by providing stronger geometric constraints for triangulating 3D points and reducing accumulated drift in the trajectory, which is especially beneficial for subsequent Bundle Adjustment.

3.2.2 Matching and Filtering Efficiency

This section quantifies their performance in identifying and retaining geometrically consistent features for robust motion estimation. We analyze two key sets of matches:

'Common Matches' - features successfully corresponding across all four relevant images (left and right of the previous and the current stereo pairs) and serve as input for motion estimation.

'RANSAC Supporters (Inliers)' - geometrically consistent correspondences identified by RANSAC from the common matches. A supporter's 3D point projection must fall within a 2-pixel threshold of its matched pixel locations in all four images when transformed by the estimated motion.

Figure 8 presents the counts of 'Total Left-Left Matches' (Matches between left images of consecutive frames), 'Common Matches', and 'RANSAC Supporters (Inliers)' across frames, illustrating the progression of features through the matching and filtering process. The graph clearly shows a progressive reduction in the number of features at each stage. The Total Left-Left Matches line is consistently the highest, followed by the Common Matches line, and finally the RANSAC Supporters line, which represents the most reliable set of features. The number of RANSAC Supporters directly indicates the reliability of the features for accurate motion estimation, and the figure's trend shows that the system effectively filters a large set of initial matches down to a smaller, high-quality subset. A consistently high count of RANSAC supporters is critical as it ensures a sufficient number of reliable features are available for the PnP algorithm to perform robust motion estimation in each frame, in that way directly impacting the quality of the initial trajectory estimation.

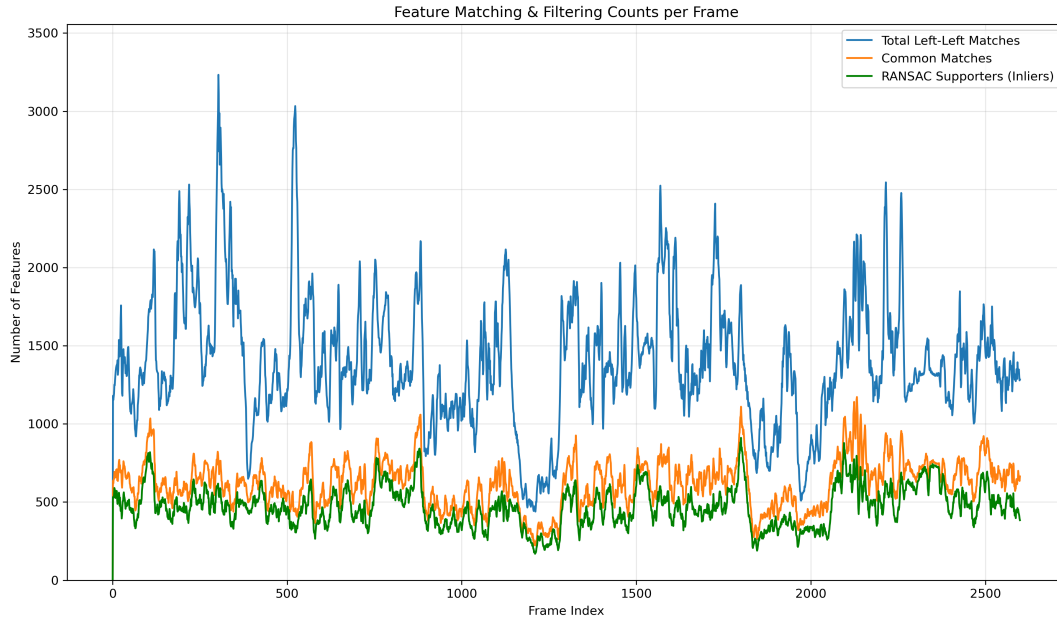


Figure 8: Feature Analysis: Average Inliers (ransac_pnp_analysis.py, line 198).

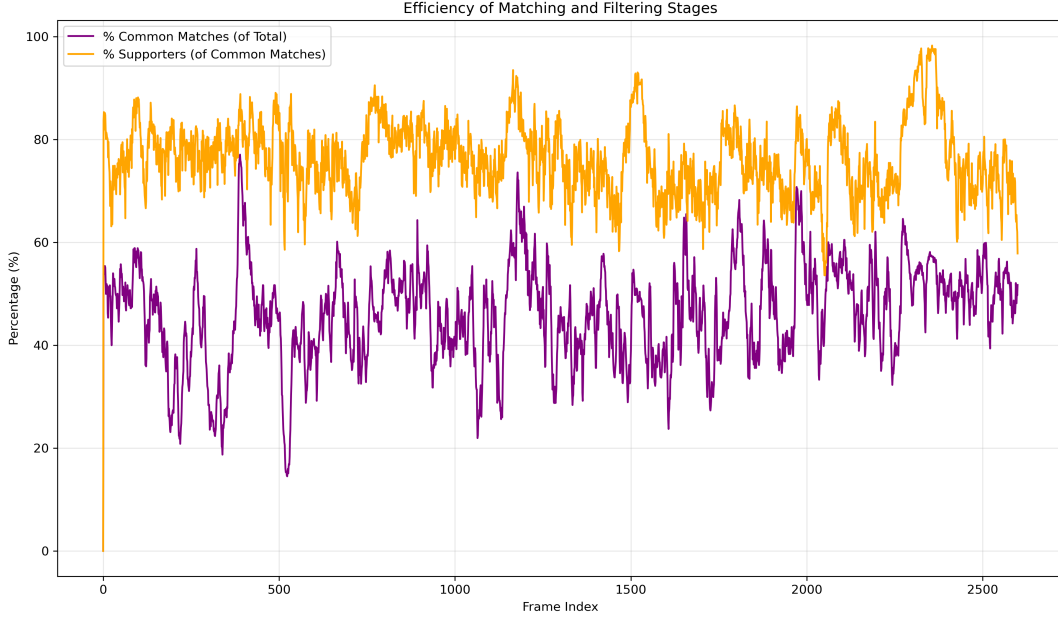


Figure 9: Efficiency of Matching and Filtering Stages (ransac_pnp_analysis.py, line 198).

Furthermore, Figure 9 highlights the efficiency of these critical stages by plotting percentages over time: The percentage of common matches (of total left-left matches) and the percentage of RANSAC supporters (of common matches).

The line for RANSAC Supporters is consistently high and appears stable, particularly in contrast to the greater fluctuation observed in the Common Matches line. This indicates that a high proportion of the features passed into the RANSAC algorithm are determined to be inliers (geometrically consistent). This high success rate demonstrates that the features filtered prior to this stage are of high quality, providing a sufficient number of inliers for a reliable and robust motion estimation. The line representing the Common Matches is notably lower and shows greater fluctuation. The graph reveals several distinct periods of significant drops in this metric, particularly in the frame ranges around 180-220, 300-320, and 510. These drops are evidence that the system encountered challenging conditions such as occlusions or rapid camera motion (the driver turning left or right, etc.), where it struggled to find a high percentage of features that are consistent across all four images.

3.3 Trajectory Estimation Performance

Figure 10 presents a bird’s-eye view comparison of different trajectory estimation methods: PnP, Bundle Adjustment, and Pose Graph optimization (with loop closure), alongside the ground truth trajectory for reference.

From the figure, we can observe that the PnP method provides a reasonable initial estimation. This trajectory is further refined through Bundle Adjustment, and then improved even more by the Pose Graph optimization with loop closure, which most closely aligns with the ground truth.

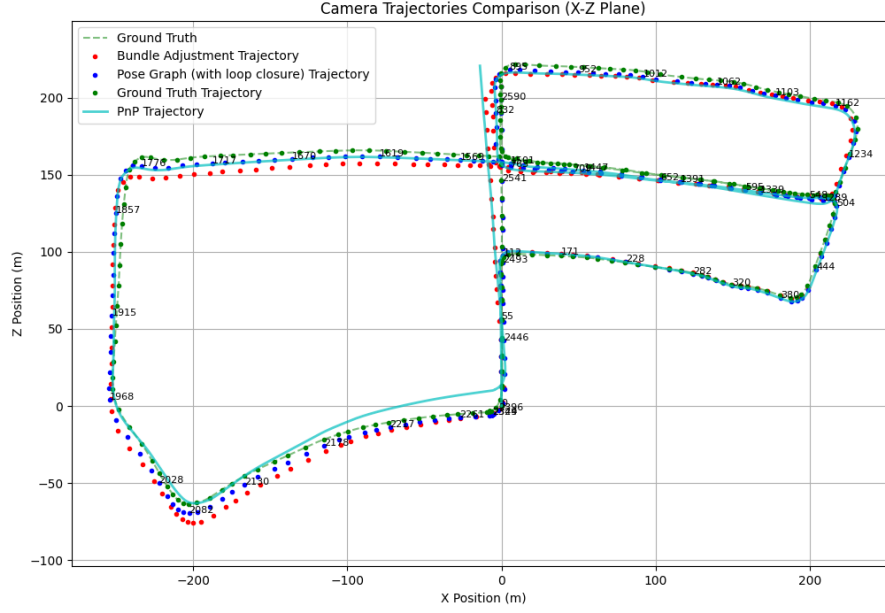


Figure 10: Comparison of estimated trajectories (PnP, Bundle Adjustment, Pose Graph with and without Loop Closure) against ground truth, viewed from a bird’s eye perspective. (plot_trajectories.py line 103)

3.4 Bundle adjustment optimization analysis

3.4.1 Optimization Error - Mean factor vs keyframes

In the Bundle Adjustment step, we construct a separate graph for each window and optimize them individually. Figure 11 shows the mean factor error for each window before and after optimization. From this figure can draw two conclusions:

- First, the initial errors are within reasonable bounds, indicating that the initial estimates are reliable, the relative pose calculations within each window’s coordinate system are accurate, and the PnP estimates are sound.
- Second, the mean factor error consistently decreases after optimization across all windows, demonstrating that the optimization process effectively refines the solution.

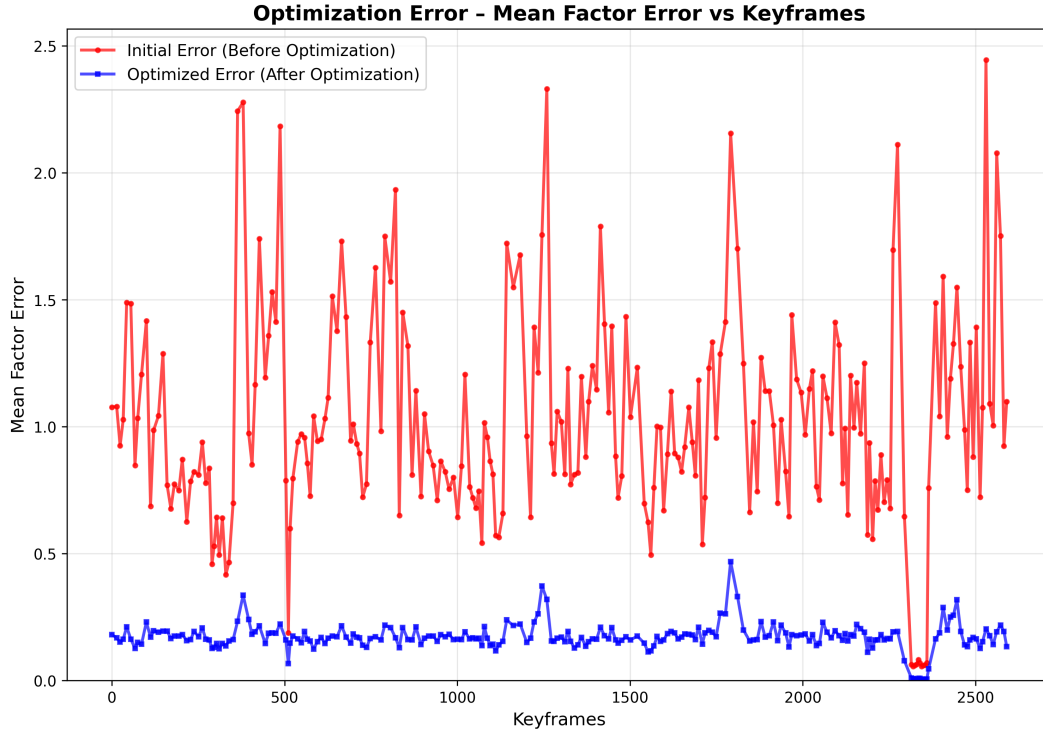


Figure 11: Mean factor error of bundle adjustment windows before (initial) and after (optimized) optimization, as a function of keyframes (plot_optimization_errors.py line 112)

3.4.2 Median Projection Error

Figure 12 shows the median reprojection error for Bundle Adjustment plotted against keyframes, both before and after the optimization. From the plot, we can see that before optimization, the reprojection error is generally higher and fluctuates significantly between keyframes. After optimization, the error drops to lower values and becomes much more stable and consistent across the trajectory. This result aligns with the trend observed in the mean factor error in Figure 11, which is expected because there is a direct relationship between the factor error in the graph and the resulting reprojection error. In other words, as the optimization reduces the factor errors, the accuracy of the keyframe poses improves, leading to lower and more consistent reprojection errors overall.

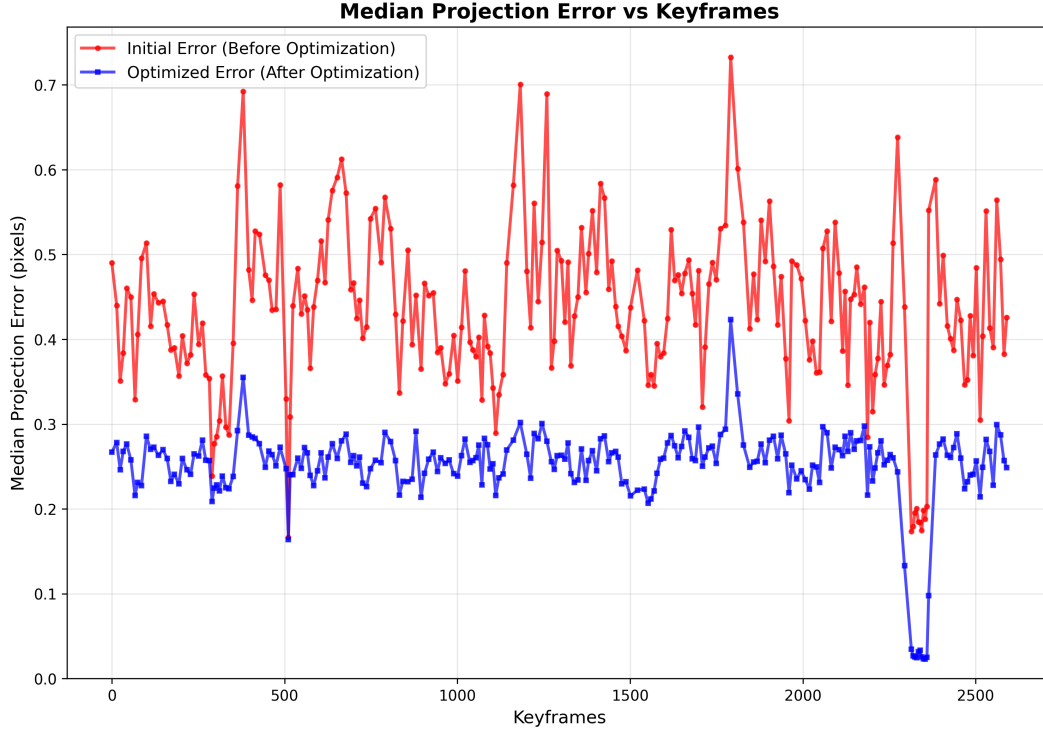


Figure 12: Median projection error of bundle adjustment windows before (initial) and after (optimized) optimization, as a function of keyframes (plot_median_projection_errors.py line 204)

3.4.3 Projection Error vs. Distance from Reference Frame

To demonstrate the impact of the optimization, in figure 13 we examine the reprojection errors as a function of the distance from the reference frame. For each track in the PnP phase, the reference frame is defined as the last frame of the track, while for Bundle Adjustment (both pre- and post-optimization), it is set as the first keyframe of the window. The plots show the mean and standard deviation of the reprojection errors. In the PnP results, the reprojection error increases with distance from the reference frame, illustrating the cumulative triangulation error that builds up during each PnP step. Before optimization, the reprojection error actually decreases with distance because the initial estimate for each track comes from triangulating using the last frame; as we move closer to this frame, the projections align better with the initial guess. After optimization, we observe two clear effects: first, the mean and standard deviation of the reprojection error are lower across all distances, confirming that the optimization effectively refines the estimates. Second, the mean reprojection error remains consistent regardless of distance, indicating that the optimizer balances the entire window uniformly rather than favoring frames near the reference.

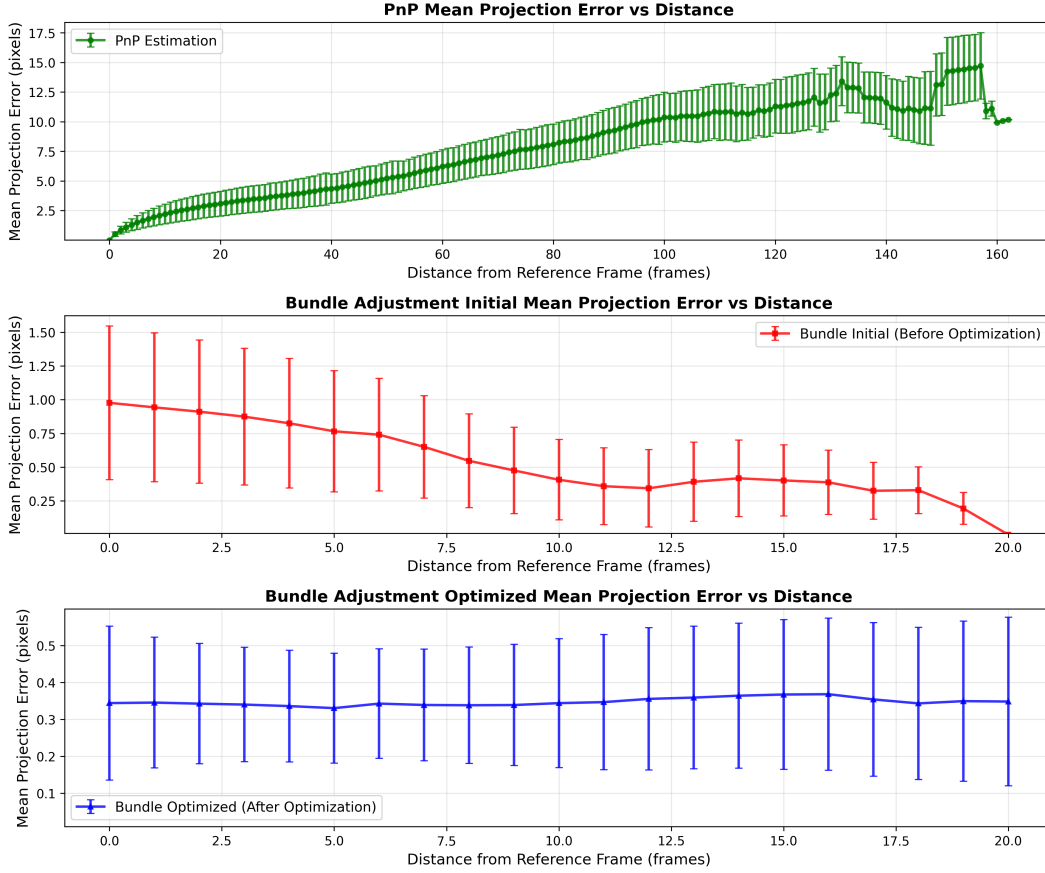


Figure 13: Mean and std projection error of track links as a function of distance from the reference frame for PnP and Bundle Adjustment estimations (plot_projection_vs_distance.py line 328)

3.5 Absolute Estimation Error

Figures 14 and 15 below compare the absolute translation and rotation errors of three different approaches: PnP, Bundle Adjustment, and Pose Graph with loop closure.

From the translation error plots, it is evident that the Pose Graph method consistently achieves the lowest errors across all axes (X, Y, Z) and in the total translation norm, indicating its effectiveness in reducing drift, especially when loop closures are used. Bundle Adjustment shows moderate improvement over PnP but does not match the Pose Graph’s performance, particularly towards the end of the sequence where the PnP method’s errors grow significantly. The rotation error comparison also confirms this trend: PnP accumulates large rotation errors over time, while Bundle Adjustment reduces these errors to a stable level, and the Pose Graph achieves the lowest rotation errors throughout the sequence. Overall, these results demonstrate that incorporating global optimization with loop closures significantly improves both translation and rotation accuracies.

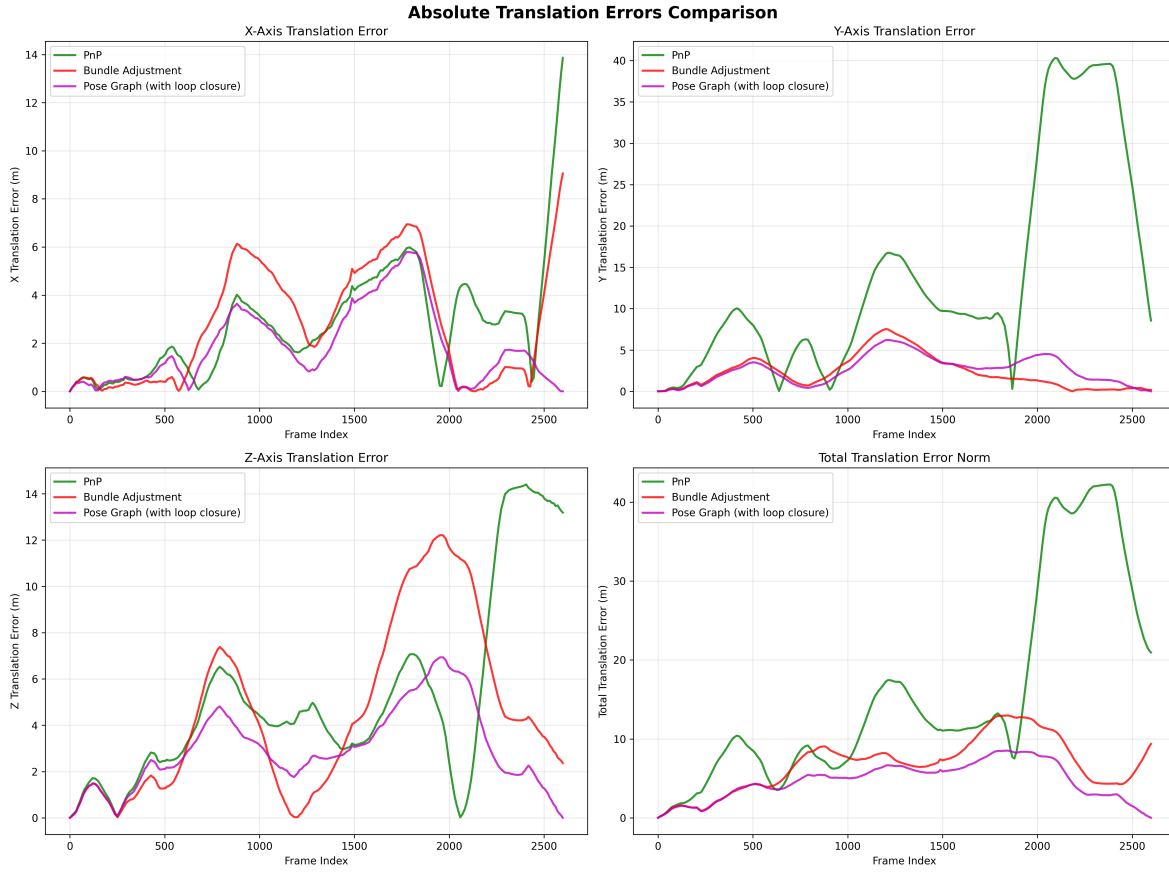


Figure 14: Absolute translation error (meters) distribution (plot_absolute_errors.py line 363).

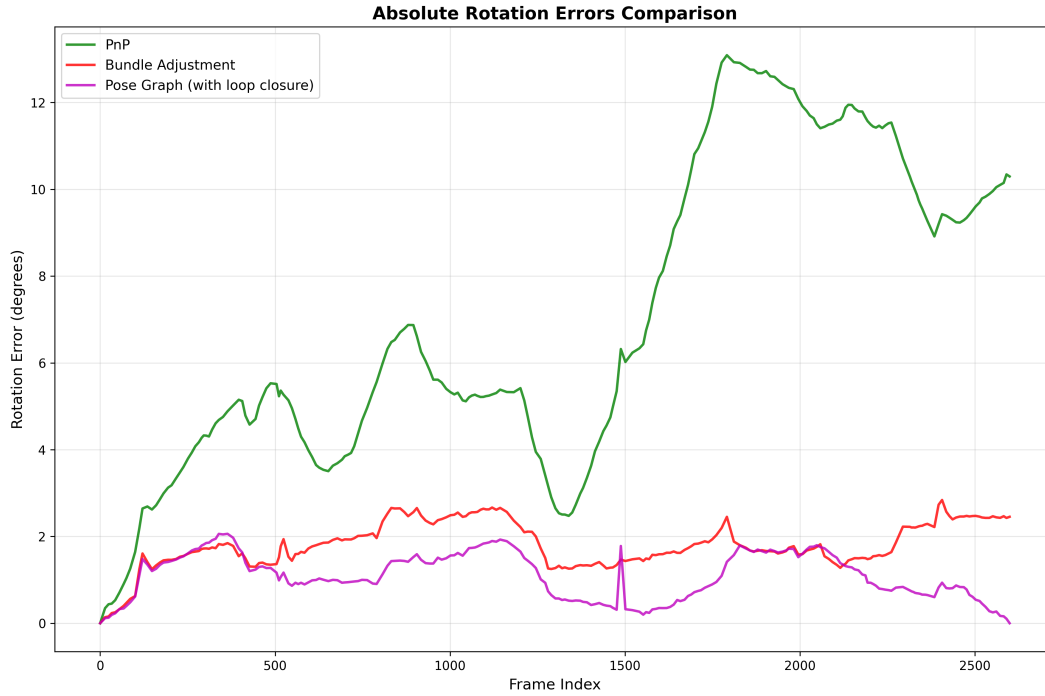


Figure 15: Absolute rotation error (degrees) distribution (plot_absolute_errors.py line 363).

3.6 Relative Estimation Error

Splitting the trajectory for PnP and Bundle Adjustment breaks the global consistency, making absolute errors less meaningful. Relative errors better reflect how well the local motion is estimated between frames, which is crucial for comparing methods that optimize parts of the trajectory independently.

We measure the relative distance and angle errors as a proportion of the total distance traveled between two poses. These errors are evaluated both frame by frame and over different sequence lengths (100, 400, and 800 frames).

In figure 16 we compare the relative errors in consecutive frames. In this figure we observed unusual spikes to extremely high values between frames 2300 to 2400. This suggests that the ground truth poses for these frames are likely inaccurate, leading to unrealistic relative error measurements.

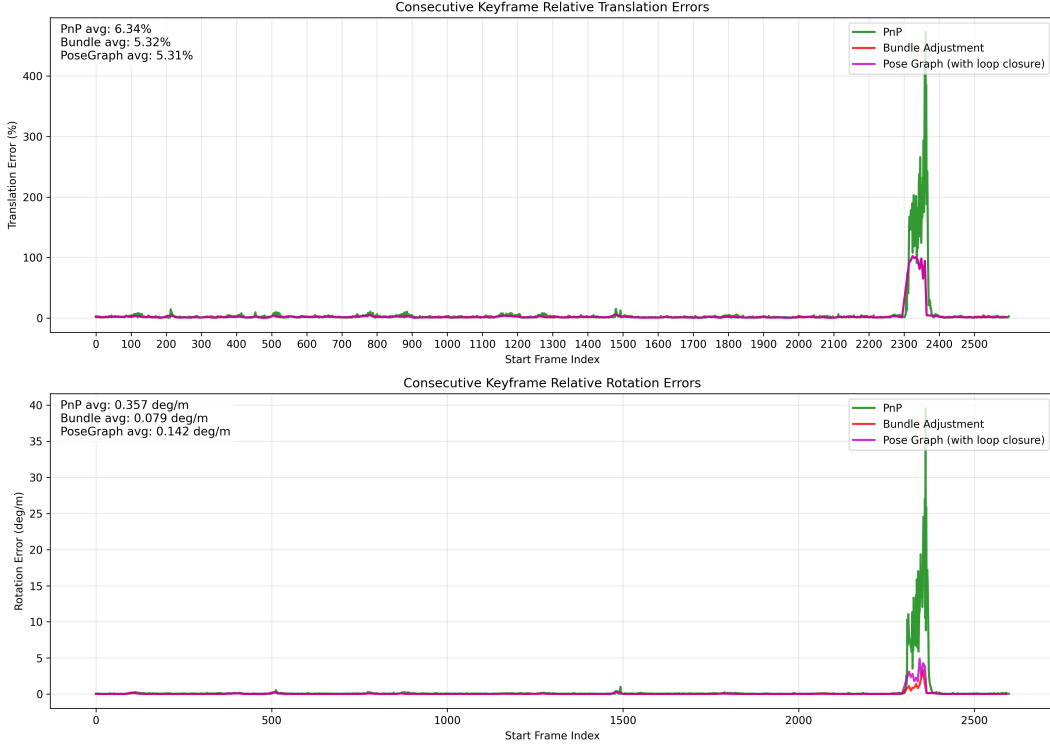


Figure 16: Consecutive relative error over the whole track (plot_relative_errors.py line 466).

To better examine the relative error, in figure 17 we plotted the same relative errors values as above but only until frame 1300. The results highlight that while PnP exhibits larger fluctuations and higher peaks in both relative translation and rotation errors, Bundle Adjustment significantly smooths these local errors by refining keyframe poses. The Pose Graph method achieves the lowest and most stable relative errors overall, demonstrating that incorporating loop closures helps maintain local consistency between consecutive frames. This confirms that global optimization not only reduces drift but also preserves accurate local motion estimation throughout the sequence.

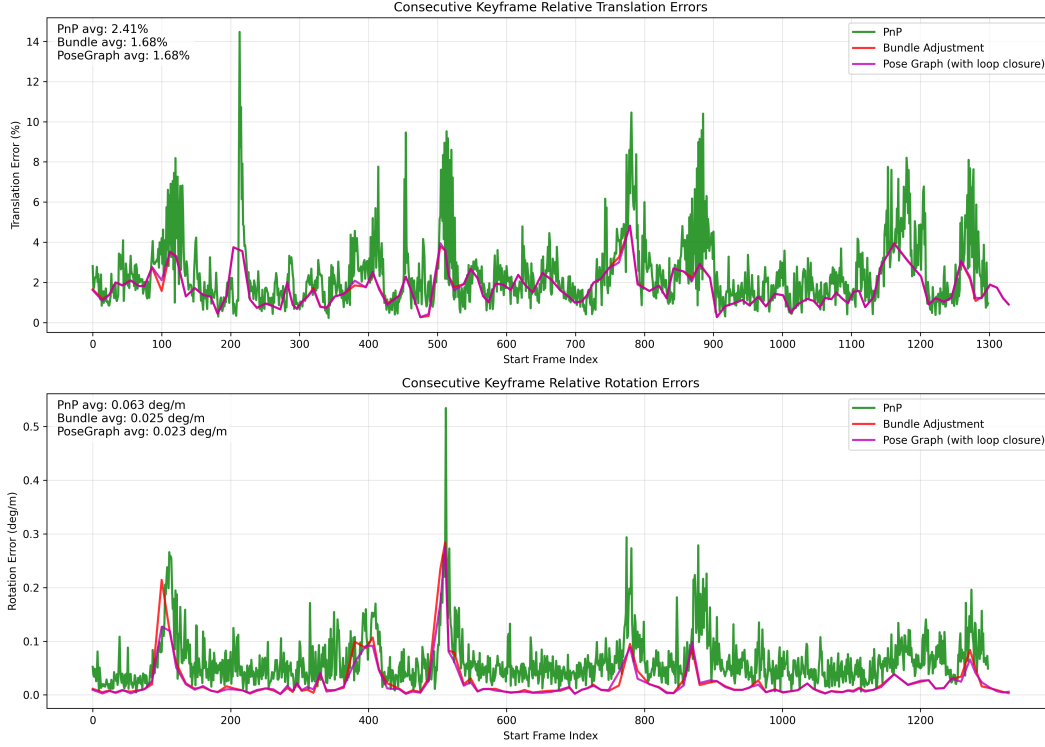


Figure 17: Consecutive relative error between frames 0 to 1300 (plot_relative_errors.py line 490).

Examine longer sequence lengths at figures 18, 19 and 20 highlights the effect even more clearly. For short segments (e.g., 100 frames), the differences between methods are smaller because drift has less time to accumulate. As the segment length increases to 400 or 800 frames, errors from PnP grow much more noticeably, showing how drift accumulates without global corrections. Bundle Adjustment helps reduce this drift by optimizing locally, but since it still works in separate windows, some accumulated drift remains. In contrast, the Pose Graph performs best over long segments because loop closures link the entire trajectory together, enforcing global consistency and keeping drift minimal even across large distances.

Overall, these results show that the Pose Graph provides the most accurate and consistent trajectory, while PnP alone produces the largest errors due to its lack of global refinement. Comparing different segment lengths demonstrates how well each method handles drift over time: the longer the segment, the more it tests the system’s ability to stay globally consistent — an area where loop closures clearly make the biggest difference compared to the other methods.

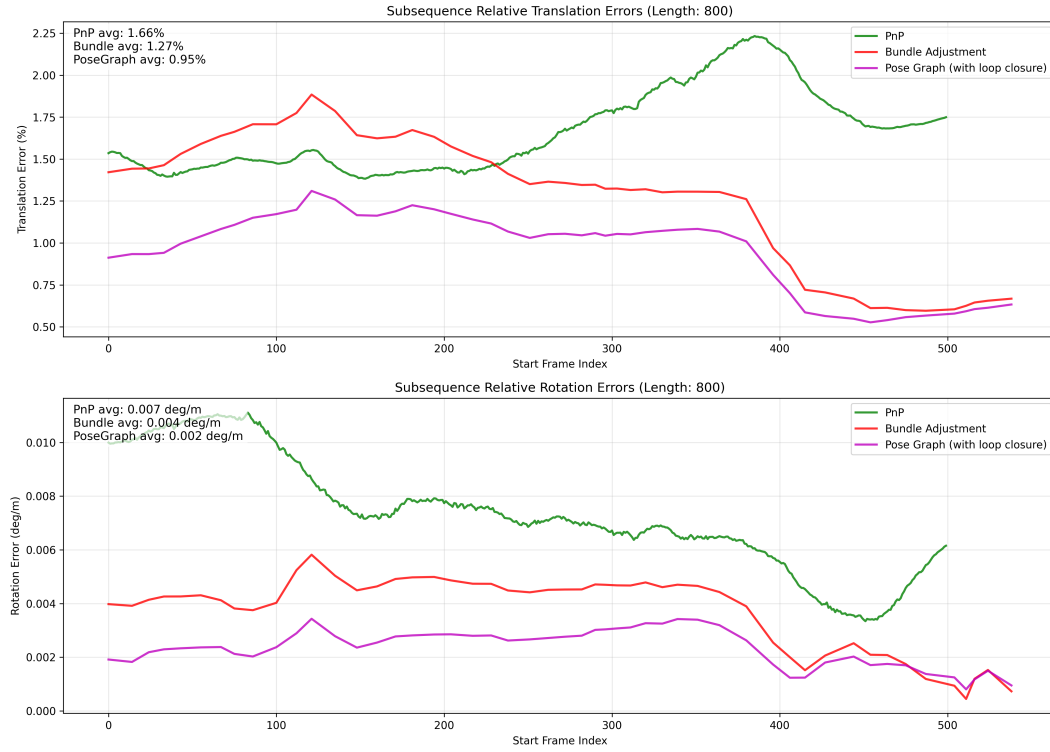


Figure 18: Relative error with 100 frames sequence, (plot_relative_errors.py line 506)



Figure 19: Relative error with 400 frames sequence. (plot_relative_errors.py line 506)

Figure 20: Relative error with 800 frames sequence. (plot_relative_errors.py line 506)



4 Discussion and Conclusions

1. Overall, the trajectory estimation performs well. Even using just PnP, the maximum relative error for frame by frame is $14m/m$ while the other methods reduce this to around $5m/m$, demonstrating a clear improvement with global optimization.
2. During the project, we encountered several points where we had to manually select hyperparameters for our implementation—for instance, minimum and maximum disparity (in the y and x directions accordingly), success probability, and the maximum distance for support point selection. While we understand that these hyperparameters can significantly influence the estimation results, we did not conduct a sensitivity analysis to assess their impact. As a result, we cannot be certain whether the chosen values were optimal, and acknowledge this as a potential weakness in our approach.
3. We conducted a comprehensive analysis of feature detector performance, including investigating and exploring new metric to compare between detectors such as spatial distribution and repeatability scores. However, identifying the most effective detector remains challenging, as its impact is tightly coupled with the performance of the entire SLAM pipeline. It is difficult to define a single metric that fully captures end-to-end effectiveness. Future work could explore the relationship between detector-descriptor combinations and the resulting pose estimation accuracy, as well as develop more robust evaluation criteria for feature detection in SLAM systems.
4. In many real life SLAM application usage you need to take in account time complexity, because the trajectory calculation is done online, which we ignored in our project and calculated the trajectory offline.
5. In order to avoid numerical instability during the marginal covariance computation, we removed keypoints with large depth (high Z values). However, these keypoints carry important information for accurate trajectory estimation, so removing them may degrade the results. A better solution would be to develop a more robust marginal covariance calculation that can handle such points without causing numerical issues.
6. Incorporating real loop closures earlier and throughout the trajectory, rather than only at the end, would likely improve global consistency and reduce drift even further.
7. The system assumes normally distributed errors, which is an unrealistic simplification. Exploring more accurate noise models could lead to improved estimation results.
8. During the bundle adjustment, we assigned the same identity covariance to all poses and the same identity covariance for all landmarks, which does not reflect the true geometric uncertainty. Future work could look into adapting covariances based on geometric relationships or measurement quality to better capture the system’s confidence in different parts of the map and trajectory.
9. Finally, beyond the technical results, this project was a rewarding learning experience - we gained valuable insights into visual SLAM and practical optimization techniques.